# Robotic Ultra-Long-Horizon Manipulation Skills via Human-guided Lifelong Code Generation

Yuan Meng[1,*], Zhenguo Sun[1,2,3,*,†], Max Fest[1], Xiangtong Yao[1], Xukun Li[1,2,3], Zhenshan Bing[4,#], and Alois Knoll[1] *Fellow, IEEE*

arXiv:2509.18597v3 [cs.RO] 7 Oct 2026

***Abstract*—Large language models (LLMs) can translate natural-language instructions for robotic manipulation into executable code, but ambiguity, noisy generations, and limited context windows make ultra-long-horizon tasks unreliable. Closed-loop approaches that rely only on LLM feedback also struggle because LLMs have limited robotic reasoning, even when task errors are obvious to humans. Feedback is often stored in representations that generalize poorly to unseen tasks and can cause catastrophic forgetting as new corrections accumulate. We propose LYRA, a human-guided lifelong skill learning and code generation framework that distills human feedback into modular, reusable skills and incrementally extends their functionality across successive interactions while preserving previously learned behavior. External memory stores learned skills and execution examples; retrieval-augmented generation selects relevant knowledge, while user hints guide reuse when retrieval is insufficient, supporting ultra-long-horizon execution. Experiments on Ravens, Franka Kitchen, LIBERO-long, MetaWorld, and real-world tasks show a 0.93 success rate, up to 27% higher than baselines, and a 42% improvement in correction efficiency. LYRA also robustly solves "build a house", which requires planning over 20 primitives.**

***Index Terms*—Agent-Based Systems, Manipulation Planning, Human-Robot Collaboration, Continual Learning**

## I. INTRODUCTION

LARGE language models (LLMs) and vision-language models (VLMs) bring commonsense knowledge and reasoning to robotic manipulation. Early approaches [1], [2], [3] used language embeddings in reinforcement or imitation learning to align robot actions with human commands, but had limited data efficiency and generalization. Later systems use LLMs such as GPT for task decomposition and language grounding [4], [5]: an LLM splits a complex task into sub-tasks and calls a pre-trained language-conditioned policy for low-level primitives. This separates high-level planning from control, but assumes that the policy can execute every requested motion precisely despite environmental perturbations and imperfect policy design. Another direction uses LLM or VLM backbones to pretrain end-to-end Vision–Language–Action (VLA) models on robotic data [6]. Robotic data are much scarcer than data for computer vision or natural language, making large-scale pretraining difficult. These models often have at least 7B parameters, require substantial compute, and are difficult to deploy on edge devices. These constraints have slowed progress. Well-pretrained LLMs can instead follow instructions and directly generate code for robot control.

LLM-generated code for robotic manipulation [7] offers another route: code itself represents the robot's actions and can draw on an LLM's instruction-following ability. Code-as-Policies (CaP) [8], [9], for example, translates human instructions into executable Python code using fixed perception and control primitives, allowing rapid deployment of new behaviors. Generated code can then be tested and revised during interaction. Yet three limitations remain: (1) LLM outputs are noisy and error-prone in long-horizon planning; (2) ambiguous language can obscure the user's intent; (3) pre-defined primitives and handcrafted prompts restrict the agent, while limited context windows constrain the number of examples. Together, these limitations hinder complex long-horizon tasks: errors compound as plans lengthen, while a fixed primitive set and a finite prompt limit adaptation. Feedback-based approaches [10], [11], [12] improve robustness, but verifiers must repeat the same corrections even for previously seen tasks, so the system has not retained what it learned. Dynamic prompt adjustment can support LLM in-context adaptation, but useful corrections must first be represented and retrieved. This raises two questions. First, **how should feedback knowledge be stored and reused?** A correction should remain useful on related tasks without asking a verifier to repeat it. Direct prompt updates [13] can cause catastrophic forgetting, sharply reducing performance on earlier tasks as new instructions are added. Cross-task insights [14], [15] often remain ambiguous and task-specific, limiting generalization to unseen tasks. Flat code is easy to modify, but repeated edits introduce noise and errors that destabilize long-horizon generation. Encapsulating temporally extended behaviors as **skill functions** offers a more stable representation: parameters can be adjusted for new tasks without repeatedly rewriting the behavior, providing flexibility and generalization while preserving stability. Second, **which feedback is most efficient**

Manuscript received: May 4, 2026; Revised July 27, 2026; Accepted September 29, 2026.

This paper was recommended for publication by Editor Aniket Bera upon evaluation of the Associate Editor and Reviewers comments.

*Equal contribution. †Project leader. #Corresponding author: bing@nju.edu.cn.

[1]Yuan Meng, Zhenguo Sun, Max Fest, Xiangtong Yao, Xukun Li, and Alois Knoll are with School of Computation, Information and Technology, Technical University of Munich, 85748 Munich, Germany. (email: y.meng@tum.de; hitsunzhenguo@gmail.com; max.fest@tum.de; xiangtong.yao@tum.de; xukunli0221@gamil.com; k@tum.de)

[2]Zhenguo Sun and Xukun Li are also with Beijing Academy of Artificial Intelligence (BAAI), 100084 Beijing, China.

[3]Zhenguo Sun and Xukun Li are also with XYZ Embodied AI, 100080 Beijing, China.

[4]Zhenshan Bing is with State Key Laboratory for Novel Software Technology, School of Intelligence Science and Technology, Nanjing University, 215163 Suzhou, China. (email: bing@nju.edu.cn)

Code is available at: https://ghiara.github.io/LYRA/.

Digital Object Identifier (DOI): see top of this page.

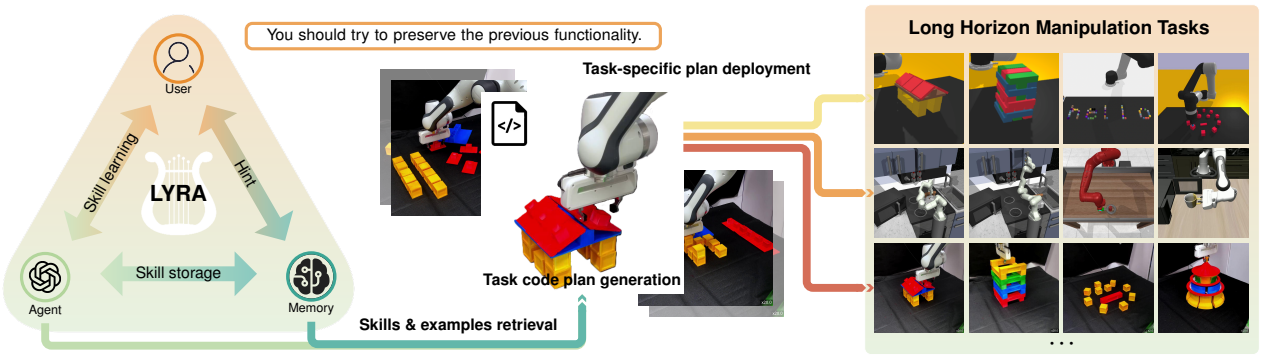


Fig. 1. Framework overview: human-guided lifelong skill code generation and ultra-long-horizon task deployment.

**during interaction?** Language is a convenient interface, but feedback generated solely by LLMs [13], [16], [12] is often unreliable: it may diverge from human preferences or fail on ultra-long-horizon reasoning [16]. Humans can usually identify, evaluate, and correct robotic task errors that an LLM may miss. In robotic code generation, those corrections can be tested and verified within seconds, whereas training conventional deep learning models often takes hours or days. This short feedback cycle makes **human-guided robotic skill learning** both reliable and efficient.

We therefore propose a human-guided framework that turns user feedback into reusable skills and extends their functionality through a user-designed curriculum, enabling preference-aligned lifelong learning (Fig. 1). Across interactions, the curriculum expands what a skill can do while preserving the functionality already learned. External memory stores learned skills and execution examples to prevent catastrophic forgetting and support reuse across tasks. Retrieval-Augmented Generation (RAG) selects relevant skills and examples for dynamic in-context learning during long-horizon planning, without placing the entire accumulated experience in a limited prompt. When retrieval is insufficient, user hints steer the agent toward the skills needed for the current task, improving efficiency and reducing interference from irrelevant data. Our system-level contribution is to show, theoretically and empirically, that **human feedback in robot control code generation can become persistent, generalizable, and compositional skill knowledge** rather than repeated local corrections. The framework connects feedback, skill storage, and code generation so that corrections made during one task can inform later tasks. Preserving, reusing, and accumulating this knowledge supports lifelong learning and ultra-long-horizon manipulation beyond fully automated LLM-based systems. We evaluate the framework on Ravens, Franka-Kitchen, LIBERO-long, MetaWorld, and real-world long-horizon tasks with a Franka FR3. It achieves a 0.93 success rate, up to 27% higher than baselines, and reduces correction rounds by 42% relative to LLM-based closed-loop methods. It also robustly solves "build a house", which requires planning over 20 primitives.

Our contributions are threefold: (1) **Persistent skill knowledge**: We establish a human-guided lifelong learning framework that distills feedback into stable, reusable, and compositional skills, retaining corrections across tasks while extending previously learned functions. (2) **Memory-augmented code synthesis**: External memory stores skills and execution examples; RAG and user hints retrieve, reuse, and selectively compose them for dynamic in-context learning and long-horizon planning while limiting interference from irrelevant experience. (3) **Ultra-long-horizon validation**: Experiments across Ravens, Franka-Kitchen, LIBERO-long, MetaWorld, and the real-world Franka FR3 demonstrate the reported success and correction gains, including planning and execution over 20 primitives in "build a house". We call our framework **LYRA**: a **L**ifelong learning code s**Y**nthesis framework for **R**obotic long-horizon skill **A**cquisition.

## II. METHOD

### A. Preliminaries

Code generation for robotic manipulation can be modelled as using an LLM $f$ to map a natural language instruction $l$ to a robot control code $c$, i.e., $f(l) = c$, where $f$ is termed a Language Model Program (LMP) in prior work [8]. Each code $c$ is achieved by multiple behaviours $b$; thus, $f$ can also be viewed as a mapping from $l$ to $b$. A behaviour $b \in \mathcal{B}$ denotes an evaluable semantic motion or outcome produced during robot execution, such as picking up an object. In our work, we argue that behaviours $b$ requiring no variation can be encapsulated as ***skills*** that reliably elicit these behaviours. We denote the skill space as $\mathcal{Z}$ and state parameters as $s$. Importantly, a skill $z \in \mathcal{Z}$ may also build on other skills:

**Definition 1**: A ***skill*** $z \in \mathcal{Z}$ is an executable parameterized function. Given state-dependent parameters $s$, its execution realizes a target behaviour $b \in \mathcal{B}$, denoted as $b = \mathrm{Exec}(z, s)$.

It is worth noting that although the task code $c$ generated by the agent represents a sequence of behaviours to accomplish a task, it is not equivalent to a skill. We define a ***task*** $(l, s_0)$ by a natural language instruction $l$ and an initial environment state $s_0$, which together form the main interface between the user and the agent. When a task is successfully completed with user feedback, we extend this tuple to $(l, s_0, c, s_T)$, where $c$ is the task-specific code (often flat, process-oriented) and $s_T$ is the achieved final state. In practice, $c$ may call skills $z$ to complete the task, and its execution should produce an evaluable task result. To align the skill output with the user's desired behaviour $b$, the agent adjusts the function details

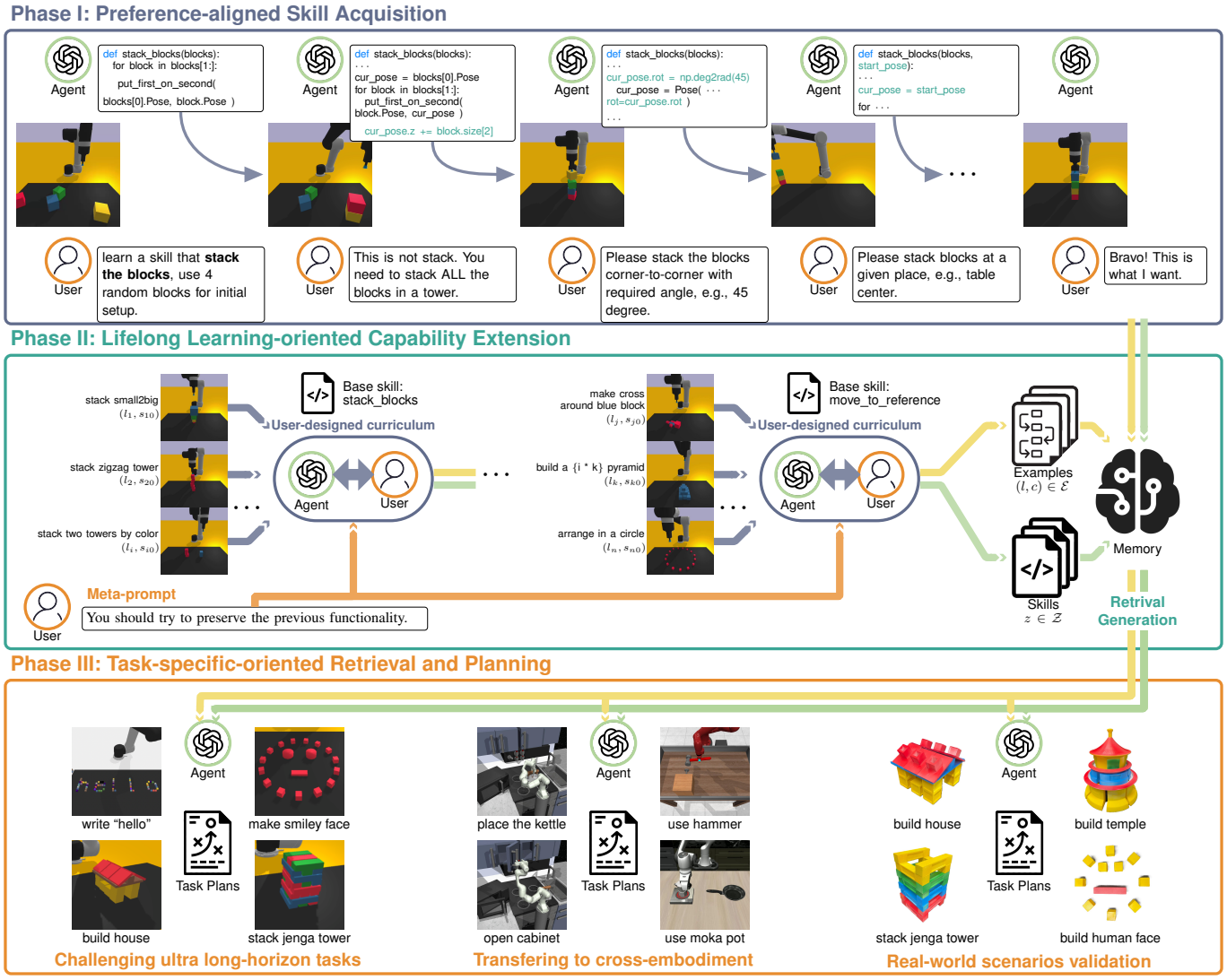


Fig. 2. Structure of the proposed human-in-the-loop lifelong skill learning pipeline.

under user guidance. We call this process ***learning the skill***. Importantly, “learning” does not refer to deep learning, instead follows a broader concept in computer science [17]:

**Definition 2**: An agent is said to ***learn*** if it produces better behavioural responses $b$ to an instruction $l$, as evaluated by a human.

Based on these definitions, the learning pipeline should answer: (1) How to encode human-provided feedback into skills that align with user preferences? (2) How to extend skills through lifelong learning while enabling dynamic in-context adaptation? (3) How to reuse and transfer learned skills to unseen long-horizon tasks?

### *B. Preference-aligned Skill Acquisition*

Based on the above questions, we design a three-phase pipeline to preserve and utilize human-guided feedback for preference-aligned lifelong skill learning in long-horizon manipulation (Fig. 2). The phases are: (1) preference-aligned skill acquisition, (2) lifelong capability extension, and (3) task-specific retrieval and planning.

To learn a new skill, three elements must be clarified: (1) what skill to learn, (2) the initial task environment, and (3) the expected functionality or behavior. Asking an LLM to infer all of this at once is impractical due to language ambiguity, so we adopt multi-turn interaction. The agent is initiated with a fixed set of skills $z \in \mathcal{Z}_0$, along with examples $e = (l, c) \in \mathcal{E}_0$ that demonstrate their use, enabling basic environment interaction. The user first describes the skill in natural language, and the LLM invokes the “skill_parse” API to generate a Python function header with suggested parameters (e.g., “def stack_blocks(blocks, start_pose):”). The user can accept or refine this skill definition. Once accepted, the LLM requests a base environment setup, and generates a corresponding task scenario. The complexity of the task is user-controlled, but overly complex tasks (over 6–8 steps) hinder skill acquisition when the agent’s ability is still limited. We therefore recommend two principles: (1) tasks should be simple, ideally completed within 1–4 primitives, and (2) tasks should allow flexibility so that one skill can adapt to diverse requirements. After the setup, the user and the LLM agent will work together to “learn” the skill. The agent generates candidate

code, deploys it in simulation, and displays results in real time. The user then accepts, rejects, or provides free-form feedback to refine the skill. User tests multiple requirements within the same scenario to check if the skill and its parameters generalize. For example, in Fig. 2 Phase I, the user guides the agent to stack blocks under different conditions such as edge alignment, tower positioning, or rotation. During this phase, **the agent can freely modify implementation details under the reserved skill name**, with the objective of producing a functional, preference-aligned skill. This forms the foundation for later phases of lifelong skill learning and capability extension.

### C. Lifelong Learning-oriented Capability Extension

Learning skills from a single task scenario is insufficient for complex ultra-long-horizon tasks, for two main reasons: (1) Simple skills cannot handle variations beyond their learning distribution. For example, a skill like "stack_blocks", which stacks four blocks at a fixed pose, cannot generalize to tasks such as building a $\{i \times j \times k\}$ structure or stacking multiple towers by color or size. (2) Directly inserting learned skills into prompts risks hallucination, where LLMs may rewrite or fabricate code during long-horizon interactions, leading to catastrophic forgetting of existing skills.

Inspired by lifelong learning, we extend the pipeline with a user-designed curriculum that enables **bottom-up skill functionality expansion** (Fig. 2 Phase II). In this phase, the agent must: (1) solve more complex user-defined task variations, (2) preserve and extend skill functionality to unseen cases, and (3) store skills permanently for reuse. The learning process mirrors Phase I, with an added **meta-prompt that explicitly asks the agent to preserve prior functionality while adapting to new tasks**. Users should design tasks outside the original distribution and aligned with long-term goals, guiding the agent to acquire foundational skills for future complex tasks. The agent can either (1) create a new named skill that calls existing ones as nested functions, or (2) extend the current skill using modularized code (e.g., if-else or match-case) to avoid overwriting. After adapting to new tasks, the agent is re-evaluated on prior tasks; if performance is preserved, we can say that lifelong learning is achieved. To further prevent forgetting and hallucination, both learned skills $z$ and explored successful examples (including task instruction and task plan) $(l, c) \in \mathcal{E}$ are stored in **external memory** and retrieved during planning.

### D. Task-specific-oriented Retrieval and Planning

In Fig. 2 Phase III, we enable **long-horizon planning** through in-context learning, adapting general-purpose LLMs to specific tasks using two main prompt inputs: (1) few-shot task-specific **examples** $\mathcal{E} = \{(l_1, c_1), ..., (l_M, c_M)\}$ that show mappings from instructions to task-specific code plan, and (2) a set of task-agnostic **skills** $\mathcal{Z} = \{z_1, ..., z_N\}$ that the agent can call to compose the plan $c$. Due to context window limits, both $|\mathcal{E}| = M$ and $|\mathcal{Z}| = N$ are fixed. The goal is that the examples and skills are representative enough for the agent to generalize to unseen pairs $(l', c')$, assuming $l'$ reflects the human's intent and the LLM can produce the correct $c'$ from $\mathcal{E}$ and $\mathcal{Z}$.

Key to our approach is **dynamically managing the context window**. As the number of examples grows, appending all of them to the prompt is infeasible, and many are irrelevant. For example, when solving "press the button", examples about "open the drawer" add noise and overwhelm the prompt. To address this, we use an external memory module with two vector databases: one for few-shot examples $(l, c)$ indexed by their instructions, and one for skills $z$ indexed by their docstrings. We implement this using ChromaDB and compute embeddings with OpenAI's text-embeddings-3 [18]. For a new instruction $l'$, we retrieve the $K = 10$ most similar examples based on cosine similarity and append them to the prompt. We also retrieve the relevant skill headers from memory and include them in the prompt. Additionally, our framework introduces a simple yet efficient mechanism for guiding the agent with a shared free-form language format: ***hints***. Hints allow users to explicitly indicate which previously learned skill may assist with the current task, effectively triggering retrieval from the library of known behaviors. Because our skill library is developed bottom-up, while long-horizon tasks require top-down decomposition, automatic similarity-based retrieval may occasionally overlook a relevant skill, making user-provided hints a simple yet highly effective way to guide the agent toward the correct skills. As the number of skills $|\mathcal{Z}|$ and behaviours $|\mathcal{E}|$ grows, most are irrelevant, and hints provide a way for the user to direct the agent toward the correct subset. If a required sub-behaviour has not yet been learned and cannot be resolved with a hint, the failure signals the user to pause the current skill and teach the missing sub-behaviour first.

With this pipeline, our framework can tackle challenging ultra-long-horizon tasks such as "build a house" and "stack a jenga tower", which require more than 20 planning steps (Fig. 2 Phase III left). We further validate our method on widely used benchmarks, including Metaworld and Franka Kitchen, covering over 20 task variations where the framework consistently solves all tasks (Fig. 2 Phase III middle). Finally, we deploy the framework in real-world settings on a franka FR3, successfully completing diverse tasks such as building a house, and generalizing to the novel task "build a temple" (Fig. 2 Phase III right). It is worth noting that although we present the learning pipeline in three phases, **the process is not strictly linear**. Whenever the agent cannot handle the current task, the user can roll back to the skill learning or extension phase and iterate until the agent meets the requirements.

## III. EXPERIMENTS

### A. Experiment Setup

Our work focuses on tabletop long-horizon manipulation tasks. For fair comparison, we include several code and language generation baselines in our experiments. Recent work, Voyager [16], explores skill learning with LLM-based automatic feedback, but it is designed for the Java-based game "Minecraft" with simple action primitives. Adapting it for Python-based robotic manipulation is costly and impractical. Thus, we design

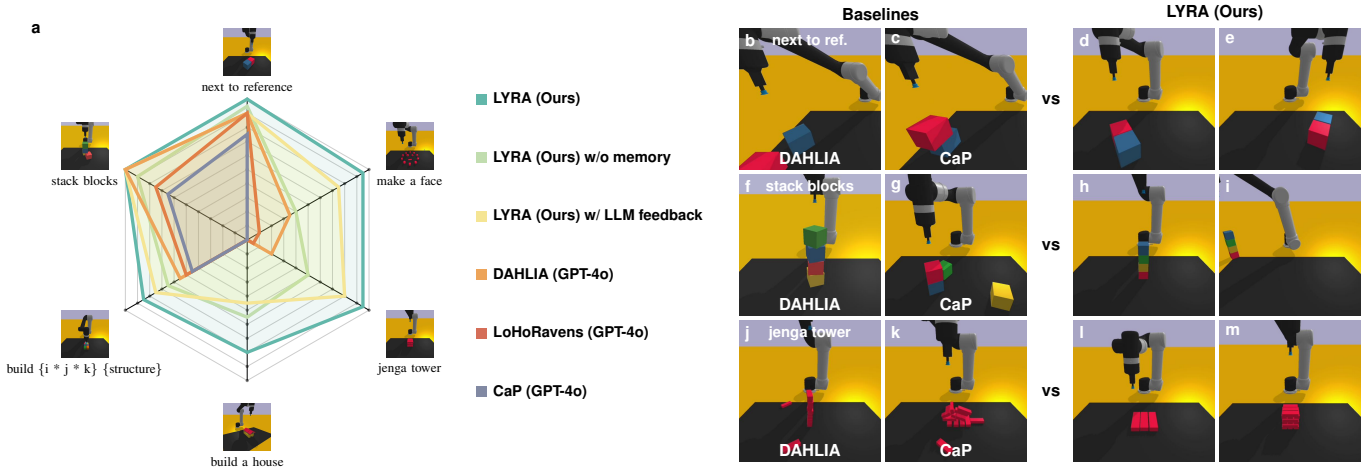


Fig. 3. Empirical analysis of skill learning. **(a)** Case study: Baseline comparison on Ravens long-horizon tasks, reported as the average success rate over 20 attempts. **(b)–(m)** Snapshots comparison: why HITL is essential for ultra long-horizon manipulation skills.

a variation of our framework that mirrors their idea, where an LLM provides feedback instead of humans for final task plan generation. We also include a w/o memory version of our framework for ablation that simulates the human-guided updates at the prompt level [13]:

**Code as Policy (CaP)** [8]: A representative baseline for LLM-based open-loop code generation without correction or retrieval.

**LoHoRavens (GPT-4o)** [4]: A language-generation baseline with explicit LLM feedback, using a pretrained RL policy CLIPort as action expert.

**DAHLIA** [12]: A state-of-the-art code generation framework with LLM-based closed-loop control and incremental examples for in-context learning.

**LYRA (Ours) w/ LLM feedback**: A variation of our framework that relies on LLM feedback, where RGB-D scenes before and after execution are given to LLM to determine task success and provide feedback. This variation has access to our well-learned memory database.

**LYRA (Ours) w/o memory**: A variant without the retrieval module, where 25 out of 49 learned skills and 25 out of 86 explored examples in Ravens are randomly sampled and appended until the context window is filled. For Franka Kitchen and MetaWorld, we sample half of the learned skills and examples at random to evaluate how irrelevant data affects feedback efficiency.

**Vision-Language-Action (VLA) Models**: To enable a comprehensive comparison, we select $\pi_0$ [6] and OpenVLA [19] as VLA baselines. For each task, we collect over 30 trajectories leveraging our LYRA output for finetuning. However, our empirical analysis finds that recent VLA models struggle on the ultra-long-horizon tasks.

For simulation, we build customized tasks upon the **Ravens** benchmark [20], [4], where a UE5 robot manipulates multiple tabletop objects (Fig. 3), including using blocks to build a house, constructing a Jenga tower, etc. Following prior work [4], we allow LLMs to set up tasks so scenarios can be modified by user intention. To test scalability, we also evaluate on **Franka Kitchen** [21], **MetaWorld** [22], and **LIBERO-long** [23]. In the real world, we deploy agents on a Franka FR3 with a RealSense camera at the table edge and validate performance on challenging long-horizon tasks. To bridge the sim-to-real gap, we adopt state-of-the-art vision foundation models [24] to obtain open-world object information.

This work involved human participants in a minimal-risk human–robot interaction evaluation and was exempt from institutional review board approval. The study evaluated robot-side performance and interaction efficiency rather than participant characteristics. All participants were university volunteers who provided informed consent. No sensitive demographic attributes were analyzed, and all interaction records were anonymized before analysis.

### *B. Human-guided Skill Learning and Reutilization*

**Ultra-Long-Horizon Performance**: To analyze how human-guided skill learning improves performance, we evaluate all models on the customized Ravens with six long-horizon tasks for a case study (Fig. 3(a)). For models with feedback, we report task plan results after up to five iterations. For CaP with open-loop control, we report the outcome of each fresh generation. All models are tested with 20 attempts, and we report the average success rate (SR) for comparison. As shown in the figure, CaP performs well only on tasks included in the prompt examples. For tasks outside the prompts, such as “build a 4 x 3 x 2 pyramid” in 3 of 5 unseen structure building tasks, it fails to generalize, achieving only a 0.45 SR. Compared with open-loop CaP, LoHoRavens and DAHLIA improve performance by using LLMs for closed-loop feedback. However, relying solely on LLM feedback can cause the agent to get stuck in long-horizon planning. Our framework variation, LYRA w/ LLM feedback, further underscores this issue. Although the agent has access to all learned skills and examples in the database, the LLM’s limited reasoning makes it uncertain about which skills to apply or what the agent can actually perform. As a result, this causes a drop in performance, and the agent may get stuck on ultra long-horizon tasks, with an average SR of only 0.77. Moreover, LYRA w/o memory simulates methods that extend data directly in the prompt. With well-learned skills and examples, this variant represents the upper bound of such methods. As shown

TABLE I
COMPARISON BETWEEN OPENVLA, $\pi_0$, AND LYRA ON LIBERO-LONG, AND ULTRA-LONG-HORIZON RAVENS TASKS.

| | LIBERO – long | Build a Jenga tower | Construct a 4×4×3 pyramid | Build a House |
|---|---|---|---|---|
| **OpenVLA** | 0.65 | 0.00 | 0.00 | 0.00 |
| **$\pi_0$** | 0.60 | 0.00 | 0.00 | 0.00 |
| **LYRA (Ours)** | 0.95 | 0.95 | 0.85 | 0.80 |

in the figure, its performance drops substantially due to the limited prompt length, and unrelated examples in the prompt further interfere with reasoning, resulting in an average SR of 0.66. By comparison, our framework achieves a high SR of 0.93, with up to **27% SR improvement** over the variant models. For a comprehensive evaluation, we also compare against recent pure learning-based end-to-end VLA models. Our framework achieves an average SR of 0.95 on LIBERO-Long, outperforming VLA models such as $\pi_0$ and OpenVLA (average 0.65). Notably, current VLAs fail on ultra-long-horizon tasks such as constructing a Jenga tower and building a house (with SR 0.0), underscoring our approach as an effective alternative for ultra-long-horizon manipulation.

**Preference Alignment and Robustness**: Relying only on success rate is limited, as it cannot capture alignment with user preferences. A key contribution of our approach is representing human preferences from language corrections in a form that can be preserved and reused through skill inheritance. Snapshots in Fig. 3(b)–(m) show how user feedback improves task performance beyond what SR reflects. For example, in (b) vs. (d), (e), and (f) vs. (h), (i), the baseline places blocks at the target positions, but their poses and distances do not match user expectations. Unlike LLM-only feedback, which often labels sub-optimal results as successful, user interventions guide the agent to acquire precise skills (e.g., stacking corner-to-corner or aligning blocks with fixed spacing) that lead to more reliable and preference-aligned outcomes. Fig. 3(j), (k) vs. (l), (m) further highlights the reliability of human-in-the-loop skill learning. In this example case, we observe **false positives** from LLM evaluation, where incomplete tasks are incorrectly marked as successful. For instance, in Fig. 3(j), DAHLIA stacks long blocks in the same direction for a Jenga tower. Although clearly incorrect to a human, the LLM still labels it as correct, raising doubts about evaluation reliability. In contrast, our framework reuses the "build jenga layer" skill, which was learned earlier with human guidance in a simpler scene, to stack a stable Jenga tower. Full results see supplementary video.

**Efficiency and Scalability**: Skill-oriented human-guided feedback is more efficient than task-specific flat code feedback from LLMs. Table II shows the average number of corrections (NoC) across Franka Kitchen (4 tasks) and MetaWorld (20 tasks). A lower NoC means the framework achieves success and aligns with user expectations more efficiently. Both DAHLIA and our LYRA variant w/o memory require more feedback iterations (4.75 and 5.00 in Franka Kitchen) compared to our LYRA framework (2.75). Although LYRA w/o memory has access to learned skills, the presence of irrelevant information in the prompt hinders reasoning and leads to more feedback iterations. In MetaWorld, DAHLIA struggles to complete several tasks within 10 feedback rounds (e.g., hammer),

TABLE II
AVERAGE NUMBER OF CORRECTIONS (NOC) FOR LLM-BASED FLAT CODE GENERATION VS OUR HUMAN-GUIDED SKILL CODE GENERATION.

| Framework | Franka Kitchen (4) | MetaWorld (MT20) |
|---|---|---|
| **DAHLIA** | 4.75 | $> 10^{\dagger}$ |
| **LYRA w/o memory** | 5.00 | 4.15 |
| **LYRA (Ours)** | **2.75** | **2.55** |

$^{\dagger}$Some tasks may not be completed within 10 feedback rounds. E.g., in the hammer task, an incorrect grasp may cause the hammer to drop, which LLM-based closed-loop feedback systems may struggle to correct, often leading to task failure.

showing the difficulty of fixing low-level execution errors through LLM feedback alone. By comparison, our LYRA framework achieves the fewest corrections in both benchmarks (2.75 in Franka Kitchen and 2.55 in MetaWorld), yielding an average **42% NoC efficiency improvement** over baselines. This gain comes not only from learning skills with human guidance, but also from our **hint** mechanism. Hints let users guide the agent using free-form text when retrieval fails or when irrelevant data is retrieved. By clarifying what the agent can do and which skills to apply, hints reduce unnecessary corrections compared to LLM feedback. We further measured the average feedback time of three human users and LLMs over 90 Ravens trials. Human users and LLMs required 27.6 s and 17.6 s on average to complete one feedback loop, respectively. Humans were faster in reasoning and decision-making (7.5 s vs. 14.2 s), while the additional human time mainly came from manual text entry; in contrast, LLMs are faster in text generation or response output (3.4 s vs. 20.1 s). Fig. 4(a)–(h) shows example snapshots of our framework's performance on three benchmarks.

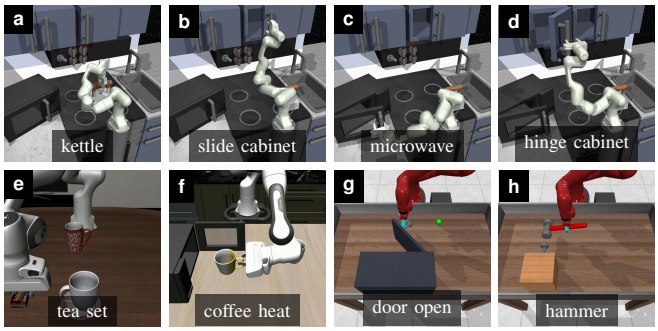


Fig. 4. Scalability validation across three benchmarks. (a)–(d) Franka Kitchen tasks: kettle, slide cabinet, microwave, and hinge cabinet. (e)–(f) LIBERO-long tea set, coffee heat task. (g)–(h) MetaWorld tasks: door open, and hammer.

### C. Can You Build a House?

To demonstrate how user-guided lifelong learning expands skill capabilities and enables challenging long-horizon tasks, we present the case study "build a house". As shown in Fig. 5(a), the task required 12 skills developed bottom-up from core primitives (orange), including 7 learned specifically for house building (light green) and others inherited from prior tasks (yellow). Starting from the chaotic initial scene (Fig. 5(b)), the task can be decomposed top-down into three subtasks: organizing the scene (Fig. 5(c)), building the base (Fig. 5(d)), and constructing the roof (Fig. 5(e)). Unlike LLM-only closed-loop methods that directly generate flat code without awareness

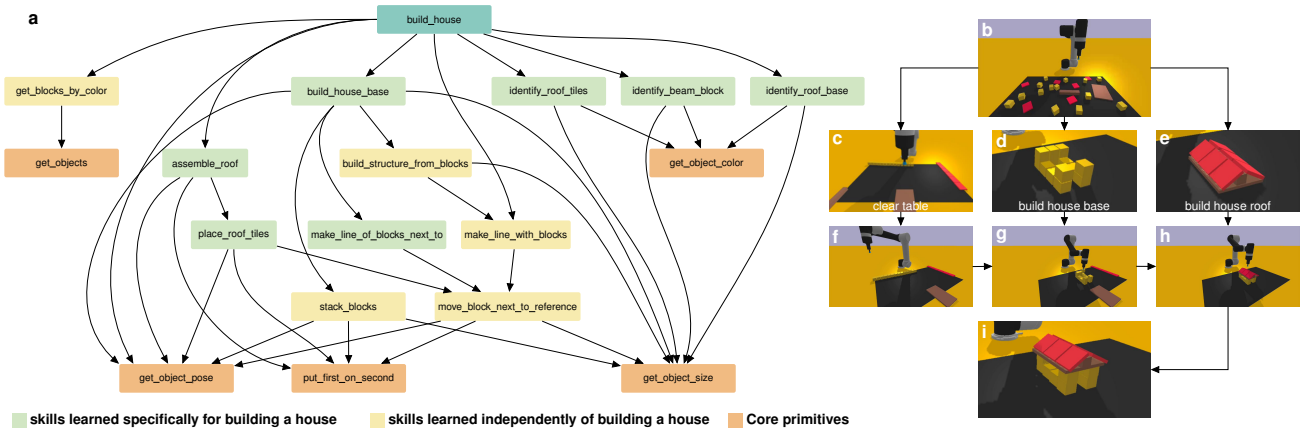


Fig. 5. Case study of "build a house". (a) Skill tree, where green nodes are house-specific skills, yellow nodes are reusable skills learned from prior tasks, and orange nodes are core primitives. (b)–(i) Task snapshots: (b) initial scene, (c)–(e) learned sub-skills for clearing the table, building the base, and constructing the roof, and (f)–(i) their reuse in the final task plan.

of required skills, our framework acquires intermediate skills such as stacking blocks corner-to-corner or aligning them with precise distance and orientation, which can be further extended to build cubes and more complex structures. Through lifelong learning, users introduce increasingly complex tasks, and the agent expands its capability by reusing existing skills, either nesting them within new ones or modularizing them to develop extended functionality. Both learned skills and examples are stored in an external database, enabling relevant information to be retrieved and appended to the prompt. This prevents catastrophic forgetting and gradually builds a complex skill tree for task planning. This integration of **top-down planning** and **bottom-up skill learning** relies on human verifiers' global awareness of which skills are necessary, a capability current LLM verifiers lack, as they tend to decompose tasks in a single direction and often get stuck in sub-optimal solutions. To the best of our knowledge, our framework is the first to successfully accomplish the "build a house" task.

### *D. Real-world Performance*

The skills learned through our pipeline are embodiment-agnostic and can be deployed on heterogeneous robot arms, enabling smooth transfer to real-world settings. We provide APIs for the Franka FR3 with consistent naming and variables to match the simulation environment. As shown in Fig. 6(a)–(d), our framework allows the agent to robustly perform challenging long-horizon real-world tasks, including building a house (0.7 SR), stacking a Jenga tower (0.7 SR), and building a human face pattern (0.9 SR). These tasks can last over 12 minutes, highlighting the framework's effectiveness in real-world deployment. We also test generalization to unseen tasks, such as "construct a temple". Within 5 NoC, the agent reorganizes the scene, builds the temple base, layers, and head by retrieving and combining existing skills, and successfully completes the task with 0.6 SR of 10 trials. Other real-world demonstrations include human preference-aligned tower stacking, where our agent stacks blocks with strictly aligned edges to achieve a human-preferred aesthetic shape. In contrast, baseline methods such as CaP fail to satisfy these requirements. More details are provided in the supplementary video.

| | build house | jenga tower | human face | build temple |
|---|---|---|---|---|
| **Success rate** | 7/10 | 7/10 | 9/10 | 6/10 |

Fig. 6. Real-world demonstrations of 10 repeat trials. Snapshots of **(a)** building a house, **(b)** stacking a Jenga tower, **(c)** building a human face, and **(d)** generalization transfer in an unseen task: constructing a temple. Full real-world results see supplementary video.

## IV. RELATED WORKS

**Code generation in embodied AI.** LLM-based code generation has shown promise by translating human instructions into executable robot-control programs [7]. CaP [8] enables language models to generate Python policies by composing perception and control primitives. More recently, CaP-X [9] provides a benchmark and engineering framework for evaluating and improving CaP agents with stronger perception, interaction, and sim-to-real components. These methods improve training-free robot code generation, but they still mainly rely on task-level program synthesis and do not explicitly preserve human feedback as reusable lifelong skill knowledge. Recent closed-loop methods further use LLMs or VLMs as task-level evaluators [4], [12], but they often struggle with ultra-long-horizon tasks because feedback remains transient and is not grounded in persistent, compositional skills.

**Robotic manipulation with language feedback and planning.** Language-conditioned systems often use LLMs for task decomposition and feedback while relying on pretrained policies or planners for execution [4], [5]. TiPToP [25] represents a recent modular open-vocabulary planning system that combines foundation-model perception with task-and-motion planning, focusing on training-free language-conditioned manipulation. Recent works [10], [13] also use human corrections to improve code generation, but their feedback is mainly stored at the prompt or data level, making it difficult to preserve and reuse preferences across long-horizon tasks. In contrast, LYRA focuses on human-guided lifelong code generation: user feedback is distilled into reusable parameterized skills, stored in external memory, and retrieved for later ultra-long-horizon planning.

## V. CONCLUSION

We introduced LYRA, a human-guided framework for lifelong skill code generation. It turns user feedback into reusable skills, preserving knowledge for ultra-long-horizon manipulation. Skill inheritance and external memory help align learning with human preferences, and LYRA outperforms prior closed-loop methods across simulation benchmarks and real-world evaluations, including the challenging task of building a house. Our current approach focuses on relatively stable manipulation primitives and standard similarity-based retrieval. Future work will extend it to contact-rich and dynamic tasks and explore multimodal, graph-based, and collaborative retrieval involving humans and LLMs.

## ACKNOWLEDGMENT

This work was supported by the Beijing Academy of Artificial Intelligence (BAAI), the NVIDIA Academic Grant Program, the National Key Research and Development Program of China (2025YFE0217000), the 2026 Innovation Project (General Program) of the National Key Laboratory for Novel Software Technology under Grant ZZKT2026A36, and the Fundamental Research Funds for the Central Universities (KG202514).

## REFERENCES

[1] X. Yao, H. Zhou, O. Mees, Y. Meng, T. Xiao, Y. Bisk, J. Oh, E. Johns, M. Shridhar, D. Shah *et al.*, "Bridging language and action: A survey of language-conditioned robot manipulation," *The International Journal of Robotics Research*, p. 02783649261468360, 2026.

[2] Y. Meng, Z. Bing, X. Yao, K. Chen, K. Huang, Y. Gao, F. Sun, and A. Knoll, "Preserving and combining knowledge in robotic lifelong reinforcement learning," *Nature Machine Intelligence*, pp. 1–14, 2025.

[3] Y. Meng, X. Yao, K. Chen, Y. Wu, L. Zhang, Z. Bing, and A. Knoll, "Pretrained bayesian non-parametric knowledge prior in robotic long-horizon reinforcement learning," *arXiv preprint arXiv:2503.21975*, 2025.

[4] S. Zhang, P. Wicke, L. K. Şenel, L. Figueredo, A. Naceri, S. Haddadin, B. Plank, and H. Schütze, "Lohoravens: A long-horizon language-conditioned benchmark for robotic tabletop manipulation," *arXiv preprint arXiv:2310.12020*, 2023.

[5] W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang, P. Florence, A. Zeng, J. Tompson, I. Mordatch, Y. Chebotar *et al.*, "Inner monologue: Embodied reasoning through planning with language models," in *Conference on Robot Learning*. PMLR, 2023, pp. 1769–1782.

[6] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter *et al.*, "$\pi_0$: A vision-language-action flow model for general robot control," *arXiv preprint arXiv:2410.24164*, 2024.

[7] K. Zhang, P. Yun, J. Cen, J. Cai, D. Zhu, H. Yuan, C. Zhao, T. Feng, M. Y. Wang, Q. Chen *et al.*, "Generative artificial intelligence in robotic manipulation: A survey," *CoRR*, 2025.

[8] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, and A. Zeng, "Code as policies: Language model programs for embodied control," in *2023 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2023, pp. 9493–9500.

[9] M. Fu, J. Yu, K. El-Refai, E. Kou, H. Xue, H. Huang, W. Xiao, G. Wang, F.-F. Li, G. Shi *et al.*, "Cap-x: A framework for benchmarking and improving coding agents for robot manipulation," *arXiv preprint arXiv:2603.22435*, 2026.

[10] J. Liang, F. Xia, W. Yu, A. Zeng, M. G. Arenas, M. Attarian, M. Bauzá, M. Bennice, A. Bewley, A. Dostmohamed *et al.*, "Learning to learn faster from human feedback with language model predictive control," *CoRR*, 2024.

[11] P. Zhi, Z. Zhang, Y. Zhao, M. Han, Z. Zhang, Z. Li, Z. Jiao, B. Jia, and S. Huang, "Closed-loop open-vocabulary mobile manipulation with gpt-4v," in *2025 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2025, pp. 4761–4767.

[12] Y. Meng, X. Yao, H. Ye, Y. Zhou, S. Zhang, Z. Bing, and A. Knoll, "Data-agnostic robotic long-horizon manipulation with vision-language-guided closed-loop feedback," *arXiv preprint arXiv:2503.21969*, 2025.

[13] M. G. Arenas, T. Xiao, S. Singh, V. Jain, A. Ren, Q. Vuong, J. Varley, A. Herzog, I. Leal, S. Kirmani *et al.*, "How to prompt your robot: A promptbook for manipulation skills with code as policies," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 4340–4348.

[14] A. Zhao, D. Huang, Q. Xu, M. Lin, Y.-J. Liu, and G. Huang, "Expel: Llm agents are experiential learners," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, 2024, pp. 19 632–19 642.

[15] L. Zha, Y. Cui, L.-H. Lin, M. Kwon, M. G. Arenas, A. Zeng, F. Xia, and D. Sadigh, "Distilling and retrieving generalizable knowledge for robot manipulation via language corrections," in *2024 IEEE international conference on robotics and automation (ICRA)*. IEEE, 2024, pp. 15 172–15 179.

[16] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, "Voyager: An open-ended embodied agent with large language models," *arXiv preprint arXiv:2305.16291*, 2023.

[17] T. M. Mitchell, *The discipline of machine learning*. Carnegie Mellon University, School of Computer Science, Machine Learning ..., 2006, vol. 9.

[18] A. Neelakantan, T. Xu, R. Puri, A. Radford, J. M. Han, J. Tworek, Q. Yuan, N. Tezak, J. W. Kim, C. Hallacy *et al.*, "Text and code embeddings by contrastive pre-training," *arXiv preprint arXiv:2201.10005*, 2022.

[19] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. P. Foster, P. R. Sanketi, Q. Vuong *et al.*, "Openvla: An open-source vision-language-action model," in *8th Annual Conference on Robot Learning*, 2024.

[20] A. Zeng, P. Florence, J. Tompson, S. Welker, J. Chien, M. Attarian, T. Armstrong, I. Krasin, D. Duong, V. Sindhwani *et al.*, "Transporter networks: Rearranging the visual world for robotic manipulation," in *Conference on Robot Learning*. PMLR, 2021, pp. 726–747.

[21] A. Gupta, V. Kumar, C. Lynch, S. Levine, and K. Hausman, "Relay policy learning: Solving long-horizon tasks via imitation and reinforcement learning," in *Conference on Robot Learning*. PMLR, 2020, pp. 1025–1037.

[22] T. Yu, D. Quillen, Z. He, R. Julian, K. Hausman, C. Finn, and S. Levine, "Meta-world: A benchmark and evaluation for multi-task and meta reinforcement learning," in *Conference on robot learning*. PMLR, 2020, pp. 1094–1100.

[23] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, "Libero: Benchmarking knowledge transfer for lifelong robot learning," in *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, Eds., vol. 36. Curran Associates, Inc., 2023, pp. 44 776–44 791.

[24] T. Ren, S. Liu, A. Zeng, J. Lin, K. Li, H. Cao, J. Chen, X. Huang, Y. Chen, F. Yan *et al.*, "Grounded sam: Assembling open-world models for diverse visual tasks," *arXiv preprint arXiv:2401.14159*, 2024.

[25] W. Shen, N. Kumar, S. Chintalapudi, J. Wang, C. Watson, E. Hu, J. Cao, D. Jayaraman, L. P. Kaelbling, and T. Lozano-Pérez, "Tiptop: A modular open-vocabulary planning system for robotic manipulation," *arXiv preprint arXiv:2603.09971*, 2026.